STATIC RESEARCH REPORT

Information Retrieval and Information Updates for Agents

A practical report on agent-ready institutional knowledge for data science and machine learning teams.

Knowledge architecture | Hybrid retrieval | Freshness and verification | Agent skills | ML repos | Evaluation

Prepared from OpenAI Deep Research
24 May 2026

Designed as a static source report for downstream LLM and HTML presentation generation.

Information Retrieval and Information Updates for Agents

Executive summary

The strongest pattern across the current tooling landscape is not a single universal “agent repo format”. It is a stack of complementary practices: metadata and ownership kept close to the source code; repository-local instruction and skill files; hybrid retrieval that combines keyword, vector, and metadata filters; validation workflows that check freshness and correctness; and evaluation systems that score not only answers but also tool use and execution traces.

For data science and ML teams, the practical opportunity in the next three to six months is not to build an all-singing all-dancing knowledge graph on day one. It is to make institutional knowledge **retrievable, trustworthy, updateable, and executable**. In practice that means: clear repo entry points; machine-readable metadata; explicit owners; freshness policies; tests or contracts for volatile knowledge; and a small set of high-value agent skills whose outputs can be reviewed and measured. Hybrid retrieval, metadata filtering, semantic code navigation, and evaluation with traces are established enough to deploy now; graph-heavy retrieval and persistent cross-session memory are promising, but should be used selectively.

The most useful operating model is simple. Use **documents** for stable human-readable knowledge, **skills** for repeated multi-step work, and **tools/resources** for live or volatile state. MCP’s distinction between resources and tools makes that especially clear: some things should be retrieved as context, while other things should be invoked as controlled operations. The mistake to avoid is treating every piece of operational knowledge as prose in Markdown. Quite a lot of it should instead be exposed as a verified command, contract, assertion, or API-backed resource.

Recommended north star:

> Every important knowledge asset should have a stable identifier, owner, freshness rule, retrieval hints, verification path, and an explicit decision on whether it is a document, a skill, or a live resource.

Once that exists, agents stop behaving like clever tourists and start behaving like slightly obsessive colleagues who can actually find the right door.

Central design question

What should a data science or machine learning team change in its repositories, documentation standards, metadata, validation workflows, and agent skills so that AI agents can retrieve, trust, update, and act on institutional knowledge reliably?

The practical answer is to redesign institutional knowledge around four linked layers:

Layer	Purpose	Examples
Content	Make knowledge understandable to humans and agents	README files, architecture docs, ADRs, runbooks, model cards, dataset cards
Metadata	Make knowledge discoverable, filterable, and governable	Owner, domain, status, freshness, source of truth, tags, stable IDs
Verification	Make knowledge trustworthy	Tests, contracts, assertions, schema checks, freshness workflows, human review
Action surfaces	Make knowledge executable	Skills, tools, MCP resources, registry/API lookups, approved commands

Landscape scan

Public evidence suggests that the market is converging on a few core patterns. The table below distinguishes between what looks established, what is becoming common, and what still deserves a health warning.

Theme	What public evidence shows	Practical reading for DS/ML teams	Evidence level
Docs and metadata near the source	Backstage's catalogue is built around metadata YAML files stored with code and maintained by owners. GitHub, Codex, and Claude Code all read repository-local instruction files.	Keep knowledge next to the thing it describes. Do not build a separate wiki first and hope the repo will somehow remain sensible.	Established

Theme	What public evidence shows	Practical reading for DS/ML teams	Evidence level
Repository-local agent guidance	GitHub supports repository-wide and path-specific instructions plus `AGENTS.md`. Codex reads `AGENTS.md` before work. Claude Code reads `CLAUDE.md`, skills, and settings from the project directory.	A repo should expose machine-readable guidance for agents in addition to human prose.	Established and fast-standardising
Metadata-rich documentation	Hugging Face model cards are Markdown files with additional metadata. Dataset cards use YAML front matter in `README.md`. dbt `meta` is compiled into `manifest.json`. Backstage and DataHub treat metadata as a first-class graph.	Markdown alone is not enough; front matter and manifests matter.	Established
Retrieval is hybrid, not purely vector	Azure AI Search states that hybrid retrieval runs full-text and vector search in parallel and merges results. LlamaIndex supports metadata filtering, metadata extraction, recursive retrieval, reranking, and small-to-big retrieval. GitHub and Sourcegraph both emphasise semantic and structural code context.	Use exact/symbol/keyword search first, then metadata filters, then vector retrieval, then recursive expansion.	Established
Verification and contracts are becoming the trust layer	DataHub assertions verify data contracts; schema assertions detect column/type changes; OpenMetadata tests monitor completeness, freshness, and accuracy; dbt contracts enforce YAML-defined schemas; Great Expectations formalises expectation suites.	Trust comes from executable checks, not from polite documentation.	Established
Evaluation is moving from prompt scoring to trace scoring	MLflow evaluates agents with traces and scorers. OpenAI's agent-evaluation guidance starts with traces and graders. LangSmith recommends defining what "good" looks like per component and beginning with 5-10 curated examples.	Measure retrieval, reasoning, tool choices, and outcomes separately.	Established
Skills as reusable institutional knowledge	Codex defines skills as packaged instructions, resources, and optional scripts. Anthropic recommends starting with evaluation and splitting bulky skills to reduce token use.	Convert repeated workflows into versioned, testable skills.	Emerging but highly practical

Theme	What public evidence shows	Practical reading for DS/ML teams	Evidence level
Graph-first retrieval	GraphRAG is aimed at corpus-level understanding and global questions, but Microsoft's own repo warns that indexing can be expensive.	Useful for discipline-wide and thematic synthesis; probably overkill as the first move for most ML repos.	Emerging
Persistent memory across sessions	LangGraph organises long-term memories under namespaces and keys. OpenAI guidance distinguishes compaction for the current run from memory for future runs.	Useful for assistants and long-running investigator agents; not a substitute for auditable documents.	Emerging

Two additional observations matter for the scope of this report. First, the repo itself is only one layer. Backstage-style developer catalogues and DataHub/OpenMetadata-style data catalogues provide cross-repo discovery, ownership, lineage, and relations that agents can exploit far better than blind multi-repo search.

Second, the public evidence is much stronger for coding assistants, data catalogues, RAG systems, and observability than for ML-specific repo operating models. That means the playbook below is a synthesis grounded in adjacent evidence, not a single off-the-shelf industry standard.

Maturity model

The maturity model below is a synthesis of current practice across repository-local instructions, catalogues, retrieval systems, validation frameworks, and agent-evaluation tooling. It is designed for DS/ML organisations rather than generic software teams.

Stage	What the team looks like	Required artefacts	Operational smell if you stay here too long
Ad hoc	Repos may be useful to humans who already know them, but agents and new joiners cannot reliably navigate them.	`README.md`, working build/test commands.	Retrieval depends on tribal knowledge and Slack archaeology.
Readable	Repos explain purpose, structure, key workflows, and ownership.	Root README, directory `index.md` pages, `CODEOWNERS`, glossary, architecture notes.	Agents can summarise, but they still guess which sources are current.
Retrievable	Knowledge is machine-filterable and searchable, not just readable.	Front matter or manifests with owner, domain, status, freshness, tags, IDs; repo-local instruction files; catalogue descriptors such as `catalog-info.yaml`.	Agents find text, but not always the right text for the right scope.
Verifiable	Important knowledge has executable checks and freshness controls.	Scheduled workflows, schema/data contracts, assertions, tests, stale-doc checks, evidence links.	Agents retrieve documents that may still be wrong, stale, or contradicted by production.

Stage	What the team looks like	Required artefacts	Operational smell if you stay here too long
Agent-operable	Repeated workflows are packaged as skills; actions are auditable; quality is measured.	Versioned skills, trace-based evals, benchmark sets, human approval gates, tool/resource permissions, update PR workflows.	Agents act, but nobody can tell whether they helped, harmed, or merely looked busy.
Institutional	Knowledge, skills, catalogues, and evaluation are shared across project, team, and discipline layers.	Cross-repo catalogue, reusable workflows, common metadata schema, central scorecards, portfolio-level KB.	You now have leverage. You may also have meetings about leverage.

The gating logic is straightforward. Do not move from **Retrievable** to **Verifiable** until critical assets have owners and review cycles. Do not move from **Verifiable** to **Agent-operable** until there are traces, scorecards, and explicit human approval points for non-trivial actions.

Recommended architecture

Design principles

The most robust design principle is to treat institutional knowledge as four linked layers: **content**, **metadata**, **verification**, and **action surfaces**. Content makes things explainable. Metadata makes them discoverable. Verification makes them trustworthy. Action surfaces make them useful to agents without copying volatile state into stale prose.

The second principle is to prefer **hierarchy before graphs**. Directory indexes, symbol-aware code navigation, metadata filters, and recursive parent-child retrieval are cheaper and easier to govern than a graph-heavy pipeline. GraphRAG becomes attractive when the question is genuinely global or multi-hop across a large corpus, not when an agent simply needs to know how to run a training job or where a feature-store schema lives.

The third principle is to keep **volatile facts live**. Table schemas, feature definitions, model versions, and production quality signals should be retrieved from resources, contracts, catalogues, or registries wherever possible. If they exist only in Markdown, agents will confidently quote last Tuesday's truth as if it were eternal law.

Reference architecture

```
GitHub-style repos
  |-- Docs and code
  |-- Repo-local metadata and instructions
         |
         v
Indexing pipeline
  |-- Keyword and symbol search
  |-- Vector index
  |-- Metadata filters
  |-- Optional graph layer
         |
         v
Agent retrieval layer
  |-- Read-only answers and reports
  |-- Skill execution
         |-- Tools and live resources
         |-- Human-reviewed outputs
         |-- Traces and evals
```

This architecture is deliberately conservative. It assumes that agents should retrieve from the cheapest and most deterministic layer first: exact file paths, symbols, keywords, filters, and only then vectors or graph summaries.

A useful rule of thumb for small, single-project contexts is that you may not need RAG at all. Anthropic notes that for a knowledge base smaller than roughly 200,000 tokens, simply passing the whole corpus can be the simplest option; once the corpus grows, a retrieval pipeline becomes the more scalable choice. Treat that as a vendor-specific heuristic, not a universal law, but it is a useful anti-overengineering test.

Reference DS/ML repo layout

For a DS/ML repo, I recommend a layout like this. This is a proposed pattern rather than a formal standard, assembled from current catalogue, instruction-file, card, and evaluation practices.

```
repo/
├─ README.md
├─ AGENTS.md
├─ CLAUDE.md
├─ catalog-info.yaml
├─ .github/
│  ├─ copilot-instructions.md
│  ├─ instructions/
│  │  ├─ modeling.instructions.md
│  │  └─ data.instructions.md
│  └─ workflows/
│     ├─ docs-freshness.yml
│     ├─ schema-verify.yml
│     ├─ skill-regression.yml
│     └─ weekly-knowledge-audit.yml
├─ docs/
│  ├─ index.md
│  ├─ glossary.md
│  ├─ adr/
│  ├─ runbooks/
│  ├─ concepts/
│  └─ experiments/
├─ metadata/
│  ├─ retrieval-manifest.yaml
│  ├─ owners.yaml
│  ├─ domains.yaml
│  └─ asset-index.yaml
├─ model_cards/
├─ dataset_cards/
├─ data_contracts/
├─ skills/
│  ├─ reproduce-experiment/
│  │  ├─ SKILL.md
│  │  ├─ inputs.schema.json
│  │  └─ eval_cases.yaml
│  └─ schema-impact-analysis/
├─ src/
├─ notebooks/
├─ tests/
│  ├─ evals/
│  ├─ retrieval/
│  └─ data_quality/
└─ mlflow/
   └─ README.md
```

The important choice is **canonical source versus vendor adapters**. Keep one canonical knowledge schema in `metadata/` and one canonical operational guide in `AGENTS.md` or a similar neutral file. Then generate or mirror platform-specific wrappers such as `.github/copilot-instructions.md`, `CLAUDE.md`, or skill packages as needed.

Example front matter for a directory index

A directory `index.md` should behave like a repo-local index page: what this directory is for, the most important entities, how to verify them, and which questions it answers. Front matter should expose the machine-readable pieces.

```yaml
---
title: Feature Engineering Index
entity_id: ds.reco.feature_engineering.index
owner: team-recommendations-ml
status: active
last_verified: 2026-05-20
review_cycle_days: 30
domain: recommendations
asset_type: index
tags:
  - feature-engineering
  - batch
  - training
questions_answered:
  - Which feature pipelines feed model training?
  - Where are freshness checks defined?
  - Which notebooks are exploratory only?
source_of_truth:
  type: mixed
  paths:
    - src/features/
    - data_contracts/
    - tests/data_quality/
verification:
  workflow: .github/workflows/schema-verify.yml
  commands:
    - pytest tests/data_quality
    - dbt test --select tag:feature_engineering
retrieval:
  keywords:
    - feature pipeline
    - training features
    - schema drift
  parents:
    - docs/index.md
  children:
    - docs/concepts/feature-store.md
    - data_contracts/features.yml
---
```

Example retrieval manifest

A retrieval manifest gives the indexing and evaluation layer a canonical place to discover what matters. This is not an industry standard today; it is a recommended operating pattern for teams that want predictable agent behaviour. It borrows from catalogue descriptors, memory namespaces, metadata filters, and skill packaging.

```yaml
version: 1
knowledge_entities:
  - id: ds.reco.model.training_runbook
    type: runbook
    paths:
      - docs/runbooks/model-training.md
    owner: team-recommendations-ml
    criticality: high
    freshness:
      review_cycle_days: 14
      verification_workflow: .github/workflows/docs-freshness.yml
    retrieval:
      strategy: hybrid
      chunking:
        mode: heading_aware
        max_tokens: 700
      filters:
        domain: recommendations
        environment: production
      synonyms:
        - training pipeline
        - model build
      questions_answered:
        - How do I retrain the model?
        - Which features and tables are required?
    action:
      linked_skill: skills/reproduce-experiment
      requires_human_approval: true

  - id: ds.reco.features.contract
    type: data_contract
    paths:
      - data_contracts/features.yml
    owner: team-data-platform
    freshness:
      verification_workflow: .github/workflows/schema-verify.yml
    retrieval:
      strategy: exact_then_hybrid
      filters:
        domain: recommendations
        asset_type: schema
```

Retrieval stack

For most DS/ML teams, the retrieval order should be:

1. **Exact and structural search first**: file paths, symbols, definitions, references, keywords, and path-specific instructions. GitHub code navigation and semantic repository indexing, plus Sourcegraph's code graph context, make this extremely useful for code-heavy repos.
2. **Metadata filtering second**: owner, domain, environment, asset type, status, and freshness. Some of the most important retrieval constraints are not in the text itself.
3. **Hybrid retrieval third**: run full-text and vector retrieval together rather than gambling on one.
4. **Contextual and recursive expansion fourth**: attach chunk context, parent summaries, or node references so that small chunks retain meaning and can expand back to larger sections on demand.
5. **Graph retrieval last, and only where justified**: use it for discipline-wide questions such as "what changed?", "what themes recur?", or "what depends on what across the corpus?", not for everyday repo navigation.

Concrete blueprints and checklists

Concrete examples by scope

Scope	Primary purpose	Must-have assets	Retrieval default	Verification loop	Typical agent outcomes
ML repo	Make code, experiments, data dependencies, and deployment assumptions navigable and reproducible	Root README, directory indexes, model card, dataset card, experiment runbook, data contracts, repo instructions, eval cases	Exact/symbol -> metadata filter -> hybrid	Scheduled test/ docs checks, schema verification, model-card review	Onboarding explanations, experiment reproduction, PR assistance, impact analysis
Project workspace	Capture one initiative end to end across code, docs, tickets, dashboards, and decisions	Project index, ADRs, goals, milestones, linked repos, output artefacts, glossary, acceptance criteria	Metadata filter by project -> hybrid retrieval	Weekly freshness review, output artefact verification, broken-link checks	Status synthesis, decision support, launch readiness review
Team KB	Provide shared standards, operating conventions, reusable runbooks, onboarding, and common skills	Team handbook, glossary, standards, escalation paths, runbooks, skill library, ownership map	Tag and owner filters -> hybrid retrieval	Monthly owner review, quarterly pruning, skill regression tests	Faster onboarding, incident support, standard-compliant code/docs
Discipline KB	Publish canonical methods, approved patterns, shared skills, domain vocabulary, and portfolio-wide relations	Cross-team standards, catalogue entities, lineage, shared metrics definitions, authoritative skills	Catalogue search + graph/relations for global questions	Governance council review, scorecards, shared benchmark suite	Portfolio synthesis, dependency discovery, discipline-wide audits

Agent-friendly repository checklist

Check	Why it matters
Root README explains purpose, entry points, build, test, and deployment	Agents and new joiners both need the first page to answer “what is this?” and “how do I verify a change?”
Every major directory has an `index.md`	This creates a navigable hierarchy instead of a bag of files.
`CODEOWNERS` exists for code and docs	Ownership is the minimum viable trust model.
Repo-local instruction files exist	Agents need project-specific expectations and path-sensitive guidance.
Knowledge assets include metadata	Owner, domain, status, freshness, and IDs make filtering and benchmarks possible.
Stable vocabulary exists	A glossary reduces retrieval drift caused by synonyms, acronyms, and legacy names.

Check	Why it matters
Authoritative cards exist for models and datasets	Model and dataset documentation improves discoverability and reproducibility.
Build, test, lint, and verification commands are explicit	Agents need executable proof paths, not merely prose.
Eval cases live in the repo	Retrieval and action quality should be regression-tested.
Sensitive paths are protected	Rulesets reduce accidental or unsafe modifications to critical files.

Freshness and verification checklist

Control	Minimum implementation
Freshness window	Every critical asset has `last_verified` and `review_cycle_days`.
Scheduled audits	Run weekly or monthly workflows to flag stale docs, unresolved TODOs, broken links, failing examples, and missing owners.
Schema drift detection	Use assertions, contracts, or tests for tables, features, and interfaces. DataHub schema assertions, OpenMetadata tests, dbt contracts, and Great Expectations all support this family of checks.
Human approval for updates	Agents raise PRs; owners approve. Do not let silent autonomous edits become canonical knowledge.
Evidence links	Every claimed fact in a critical runbook points to code, lineage, registry entry, dashboard, or assertion result.
Trace logging	If an agent proposes an update, persist the trace and evaluator outcome for audit.
Sensitive file protection	Protect contracts, skill manifests, production runbooks, and security-sensitive instructions with rulesets and code-owner review.
Update subscriptions	Where possible expose live resources and change notifications instead of copying data into docs. MCP explicitly supports resource listing, reading, and change notifications.

Documentation-to-skill framework

This framework is the heart of turning documentation into capability.

If the knowledge is…	Best form	Why
Stable, explanatory, and primarily read by humans	Document	Human-oriented context belongs in prose.
Repeated, multi-step, and testable	Skill	Skills package instructions, resources, and scripts into reusable workflows.
Volatile, live, or authoritative in an external system	Resource or tool	Live state should be fetched or invoked, not copied into stale Markdown.
Governed by schema or policy	Contract/assertion/test	Trust should come from executable validation.

If the knowledge is…	Best form	Why
Shared across many agents but still reference-like	Index + metadata + parent/child retrieval	This keeps tokens lower and retrieval more precise than stuffing everything into one prompt.
Corpus-wide and relational	Graph layer	Use this only when questions are genuinely global or multi-hop.

A good rule is this: if success can be checked automatically and the workflow repeats, it probably wants to become a skill. If not, it probably still wants to be a document. If the “truth” changes every time the warehouse or registry changes, it wants to be a live resource.

Agent skills and workflow examples

Anthropic and Codex both point towards skills as reusable packages of instructions, resources, and optional scripts. Anthropic is explicit that skill design should start from evaluation and that large skills should be split so irrelevant context does not waste tokens.

Skill	Purpose	Required knowledge	Tools/resources	Outputs	Main risks	How to evaluate
Repo onboarding analyst	Explain architecture, workflows, key entities, and known traps to a new team member	README, directory indexes, ADRs, glossary, code symbols	Repo search, symbol search, catalogue lookup	Onboarding brief with links and evidence	Hallucinated structure, stale docs	Answer accuracy, citation coverage, time-to-first-useful-answer
Experiment reproduction assistant	Reproduce a prior experiment and summarise what differs from the current pipeline	Experiment docs, MLflow runs, config files, dataset versions	MLflow, filesystem, test runner, container/ sandbox	Reproduction memo, diff, reproducibility verdict	Hidden state, missing environment assumptions	Task success, reproducibility rate, unsupported-claim rate
Schema impact analyst	Assess impact of changing a table, feature, or dbt model	Contracts, lineage, exposures, assertions, downstream owners	Data catalogue, dbt docs, schema tests	Impact report and proposed checklist	Missing downstream dependencies	Retrieval recall on dependencies, false-negative rate
Weekly knowledge freshness auditor	Find stale or contradicted knowledge and open PRs/issues	Freshness metadata, assertion results, failing checks	Scheduled workflows, link checker, assertions	PRs or issues with evidence	Noisy churn, false alarms	Precision of flags, merge rate, owner satisfaction
Model and dataset card certifier	Check whether cards are present, complete, and aligned with current artefacts	Model registry, datasets, eval reports, cards	Registry API, file reads, test outputs	Compliance report or update PR	Template compliance without substance	Completeness score, factual alignment, review rejection rate

Skill	Purpose	Required knowledge	Tools/resources	Outputs	Main risks	How to evaluate
Experiment design reviewer	Review proposed experiments before execution	Historical experiments, metrics definitions, known confounders, dataset notes	Repo context, previous memos, live metric definitions	Review memo with risks, suggested controls, success criteria	Overconfident “best practice” claims	Human win-rate against baseline process, usefulness rating

Lightweight evaluation framework

The practical consensus across MLflow, OpenAI, LangSmith, RAGAS, BEIR, RepoBench, SWE-bench, and CRAG is that evaluation has to be component-aware. Retrieval quality, answer quality, tool behaviour, and end-to-end success are related, but not interchangeable.

Dimension	Suggested metrics	Good benchmark examples
Retrieval	Recall@k, MRR, nDCG, evidence hit-rate, metadata-filter precision	“Find the owning team”, “Locate the source-of-truth schema”, “Identify the latest approved runbook”
Answer quality	Correctness, faithfulness, citation coverage, unsupported-claim rate	Onboarding questions, architecture explanations, runbook Q&A
Tool behaviour	Correct tool selection, valid arguments, safe execution, handoff quality	Schema-diff task, registry lookup task, impact-analysis workflow
Action success	Task completion, review acceptance, patch/test pass rate, human rework minutes	Reproduce experiment, update card, open verified freshness PR
Freshness	Stale-asset rate, verification pass rate, mean age of critical docs, time-to-fix stale assets	Weekly knowledge audit suite
Robustness	Performance under long context, dynamic facts, ambiguous aliases, missing metadata	CRAG-like freshness questions, “lost in the middle” stress tests, synonym-heavy retrieval cases

A sensible starting point is to define critical components, then manually curate 5-10 examples of what “good” looks like for each. From there, add larger suites. Use BEIR and RAGAS for retrieval-style thinking, RepoBench and SWE-bench for repository and coding workflows, and CRAG-inspired cases for freshness and temporal drift.

A practical internal benchmark pack for a DS/ML organisation should include at least four suites:

Suite	Purpose	Example size
Repo understanding	Can the agent explain structure, dependencies, and build/test paths?	30-50 questions per pilot repo
Data and schema trust	Can the agent identify authoritative schemas, lineage, and freshness status?	20-30 change-impact tasks
Reproducibility	Can the agent reproduce or diagnose an experiment run?	15-20 tasks

Suite	Purpose	Example size
Update and maintenance	Can the agent detect stale knowledge and propose the right update path?	20-30 audit cases

Implementation roadmap

The timeline below is intentionally conservative. It assumes one pilot ML repo, one project workspace, and one cross-team knowledge surface.

Phase	Focus	Concrete actions	Exit criteria
First 2 weeks	Inventory and selection	Pick one pilot ML repo; inventory critical docs, data assets, owners, source systems, existing pain points, and likely agent workflows.	Pilot repo selected; 20-30 real user questions collected; critical knowledge assets listed.
First month	Structure and ownership	Add or improve README, `AGENTS.md`, directory indexes, `CODEOWNERS`, glossary, and basic metadata. Identify source-of-truth conventions.	Repo is readable and navigable by a new joiner or agent; owners are explicit.
Months 2-3	Retrieval and verification	Add retrieval manifest, model/dataset cards, contracts/assertions, scheduled freshness checks, and first eval suite.	Agents can answer pilot questions with evidence; stale docs can be detected.
Months 2-3	First useful skills	Build one or two skills: experiment reproduction, schema impact analysis, onboarding, or weekly knowledge audit.	Skills produce reviewable outputs and have regression cases.
Months 4-6	Scale carefully	Extend to a project workspace and team KB. Integrate catalogue/data catalogue where useful. Standardise metadata and skill packaging.	Reusable standards exist; adoption appears in recurring team workflows.
Months 4-6	Governance and measurement	Create scorecards for retrieval quality, freshness, task success, and human review acceptance.	Leaders can see whether the system is helping, not just whether it exists.

What to avoid early

- Building a large knowledge graph before basic ownership, indexes, and metadata exist.
- Converting every document into a skill.
- Allowing agents to silently edit canonical docs.
- Treating stale Markdown as an acceptable source of truth for live schemas or production state.
- Launching a discipline-wide programme before one pilot repo has proved value.

What to automate later

- Broken-link and stale metadata checks.
- Schema-drift and data-contract verification.
- Model/dataset card completeness checks.
- Automatic PRs for low-risk documentation updates.
- Cross-repo retrieval benchmarks.
- Weekly knowledge freshness scorecards.

Signs the system is working

- New joiners find correct repo context faster.
- Agents cite the right docs and live resources, rather than plausible but wrong sources.
- Reviewers accept more agent-generated outputs with less rework.
- Stale assets are found before they cause incidents or confusion.
- Token cost per useful task falls because retrieval is more precise.
- Team members start asking for new skills because the first few are genuinely useful.

Risks and mitigations

Risk	Why it happens	Mitigation
Documentation theatre	Teams create more files but not more truth	Make verification mandatory for critical assets. Every key doc should point to a test, contract, registry entry, or live resource.
Token bloat	Agents ingest too much irrelevant context and quality drops	Split skills and instructions by path or task. Use metadata filters and recursive retrieval rather than loading everything.
Hidden authority conflicts	Wiki says one thing, code says another, warehouse says a third	Declare source of truth per asset and keep volatile state behind tools/resources.
Unsafe autonomy	Agents edit critical assets or touch credentials too freely	Require code-owner review, protect sensitive paths, and keep secrets outside execution sandboxes.
Weak measurement	Teams judge agents by vibes and anecdotes	Instrument traces, define benchmark suites, and track regressions over time.
Graph overreach	Teams jump to GraphRAG before simpler retrieval is mature	Use graphs only for global, corpus-level synthesis or multi-hop reasoning.
Skills become stale	A skill encodes an old process and keeps repeating it	Version skills, assign owners, add review windows, and run skill regression tests.
Duplicated sources of truth	Repo docs, wiki pages, dashboards, and catalogues drift apart	Use stable IDs and explicit source-of-truth fields. Prefer linking to live resources over copying volatile values.
Permission leakage	Agents retrieve context they should not see	Respect existing ACLs, limit connector scopes, and evaluate retrieval logs for inappropriate access.

Risk	Why it happens	Mitigation
Over-trust	Humans assume a well-written agent answer is correct	Require citations/evidence and mark unsupported claims. Keep human approval gates for consequential changes.

Tooling and vendor landscape

This is a practical categorisation, not a procurement recommendation.

Category	Strong current options	Best for	Notes
Repo-local guidance	GitHub Copilot instructions and `AGENTS.md`; Codex `AGENTS.md`; Claude Code project files; VS Code custom instructions and prompt files	Telling agents how to work in a specific repo	Strong convergence on repo-local guidance, but not yet one universal file format.
Developer portal and software catalogue	Backstage	Cross-repo ownership, relations, human and agent discovery	Particularly good for software and platform entities.
Data catalogue and observability	DataHub, OpenMetadata	Schema, lineage, ownership, assertions/tests, data contracts	Especially important for DS/ML teams because live data trust sits here.
Data transformation metadata	dbt	Contracts, docs, downstream exposures, metadata in manifest	Useful where analytics engineering and DS meet.
ML artefact tracking	MLflow	Experiment tracking, model registry, evaluation and monitoring	A natural bridge between repo knowledge and run history.
Retrieval framework	LlamaIndex, Azure AI Search	Metadata extraction, recursive retrieval, hybrid retrieval, reranking	Good building blocks; start simple.
Agent orchestration and memory	LangGraph/LangChain, OpenAI Agents SDK, Anthropic agent patterns, Google ADK	Memory, tool orchestration, traces, long-running agents	Choose based on existing stack and observability requirements.
Evaluation and observability	MLflow, LangSmith, OpenAI evals	Trace grading, datasets, scorers, regression loops	Use multiple layers: unit, retrieval, trace, and end-to-end.
Coding context and repo exploration	GitHub, Sourcegraph, Claude Code, Codex	Code navigation, semantic code search, repo exploration, multi-file work	Strong evidence that code-context retrieval is now core, not optional.
Protocol for live integrations	MCP	Standardising resources and tools for agents	Particularly useful for live truth such as schemas, APIs, or run metadata.

Open questions and limitations

There is not yet a single cross-vendor standard for **agent-ready repository knowledge**. Public ecosystems are converging on similar ideas - repo-local instructions, metadata, skills, and live tools/resources - but they still use

different file names and packaging models. That is why a canonical internal schema with generated adapters is safer than betting the house on one vendor's naming convention.

Evidence for **ML-specific** repo redesign is also thinner than evidence for software engineering and data-catalogue use cases. The blueprints here are therefore intentionally grounded in adjacent practices: code navigation, cards, contracts, registries, and evaluations. They are high-confidence as an operating model, but they should still be piloted and measured inside your own environment before being declared gospel.

Graph-centric retrieval deserves caution. The public evidence supports it for global corpus questions, but not yet as the default foundation for most repos. That part is still emerging, and in some places rather enthusiastically oversold.

Final recommendations

1. **Redesign repos around agent-operable metadata, not around a new documentation tool.** Put the effort into ownership, freshness, retrieval hints, and verification hooks.
2. **Treat documentation as a product and a control surface.** Model cards, dataset cards, ADRs, runbooks, and glossary pages should be discoverable, current, and linked to evidence.
3. **Separate what agents read from what agents do.** Read from indexed docs, catalogues, registries, and live resources. Act through skills and tools with approval gates, traces, and scorecards.
4. **Start with one pilot repo where failure is tolerable but value is obvious.** A production-adjacent ML repo with recurring experiment reproduction pain, schema drift pain, or onboarding pain is ideal.
5. **Deliver one or two useful skills first.** Good candidates are experiment reproduction, schema-impact analysis, onboarding, and weekly knowledge audits.
6. **Add evaluation before scaling.** A small benchmark pack with real internal questions is more useful than a large generic benchmark that nobody trusts.
7. **Scale after proof.** Move from one repo to a project workspace, then to a team KB, then to a discipline KB. Do not begin with the discipline KB. That way lies PowerPoint, and not the useful sort.

Source list

The deep research output drew on the following public sources. These references are included here so the PDF can serve as a static research artefact and as input to downstream LLM workflows.

Source	Why it mattered
Backstage Software Catalog - https://backstage.io/docs/features/software-catalog/	Source-controlled metadata, ownership, catalogue entities, and cross-repo discovery.
Backstage descriptor format - https://backstage.io/docs/features/software-catalog/descriptor-format/	Metadata schemas and catalogue descriptors.
GitHub Copilot repository instructions - https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions	Repo-local and path-specific agent instructions.
GitHub code navigation - https://docs.github.com/repositories/working-with-files/using-files/navigating-code-on-github	Structural search and code-navigation patterns.
GitHub repository indexing - https://docs.github.com/en/copilot/concepts/context/repository-indexing	Semantic repository context for coding agents.

Source	Why it mattered
GitHub CODEOWNERS - https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners	Ownership and review controls.
GitHub Actions workflow triggers - https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows	Scheduled freshness and validation workflows.
GitHub rulesets - https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets	Protecting sensitive paths and critical knowledge assets.
OpenAI Codex AGENTS.md - https://developers.openai.com/codex/guides/agents-md	Repo-local agent guidance pattern.
OpenAI Codex skills - https://developers.openai.com/codex/skills	Skills as packaged reusable institutional workflows.
OpenAI agent evals - https://developers.openai.com/api/docs/guides/agent-evals	Trace-based and component-aware agent evaluation.
Claude Code project directory docs - https://code.claude.com/docs/en/claude-directory	Project-local instructions, skills, and settings.
Claude Code best practices - https://code.claude.com/docs/en/best-practices	Context management and coding-agent workflow design.
Anthropic: agent skills - https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills	Skill packaging, evaluation-first skill design, and token control.
Anthropic: building effective agents - https://www.anthropic.com/research/building-effective-agents	Agent design patterns and effective tool use.
Anthropic: contextual retrieval - https://www.anthropic.com/news/contextual-retrieval	Contextual chunking, retrieval improvements, and small-corpus heuristics.
Anthropic: managed agents - https://www.anthropic.com/engineering/managed-agents	Separation of agent reasoning and action surfaces.
Azure AI Search hybrid retrieval - https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview	Full-text plus vector hybrid retrieval.
Azure AI Search agentic retrieval - https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview	Multi-query decomposition and retrieval orchestration.
LlamaIndex metadata filtering - https://developers.llamaindex.ai/typescript/framework/modules/rag/query_engines/metadata_filtering/	Metadata-aware retrieval.
LlamaIndex metadata extraction - https://developers.llamaindex.ai/python/framework/module_guides/indexing/metadata_extraction/	Enriching documents for retrieval.
LlamaIndex advanced retrieval - https://developers.llamaindex.ai/python/framework/optimizing/advanced_retrieval/advanced_retrieval/	Reranking, hybrid retrieval, and optimisation patterns.
LlamaIndex recursive retrieval - https://developers.llamaindex.ai/python/framework/integrations/retrievers/recursive_retriever_nodes/	Parent-child and small-to-big retrieval.
Sourcegraph Cody context - https://sourcegraph.com/docs/cody/core-concepts/context	Code graph context and repository-aware assistants.

Source	Why it mattered
Model Context Protocol resources - https://modelcontextprotocol.io/specification/2025-03-26/server/resources	Distinguishing retrievable resources from executable tools.
MCP server concepts - https://modelcontextprotocol.io/docs/learn/server-concepts	Live tools/resources and agent integrations.
DataHub assertions - https://docs.datahub.com/docs/managed-datahub/observe/assertions	Data assertions and contract-like verification.
DataHub schema assertions - https://docs.datahub.com/docs/managed-datahub/observe/schema-assertions	Schema drift checks.
DataHub lineage - https://docs.datahub.com/docs/api/tutorials/lineage	Cross-asset lineage and dependency discovery.
OpenMetadata data quality - https://docs.open-metadata.org/v1.12.x/how-to-guides/data-quality-observability/quality	Completeness, freshness, and quality checks.
OpenMetadata lineage - https://docs.open-metadata.org/v1.12.x/api-reference/lineage	Data lineage for impact analysis.
dbt contracts - https://docs.getdbt.com/reference/resource-configs/contract	Enforcing schemas from YAML-defined contracts.
dbt exposures - https://docs.getdbt.com/docs/build/exposures	Downstream dependencies and exposed assets.
dbt meta - https://docs.getdbt.com/reference/resource-configs/meta	Custom metadata compiled into manifests.
dbt documentation - https://docs.getdbt.com/docs/build/documentation	Data documentation and generated docs.
Great Expectations expectations - https://docs.greatexpectations.io/docs/core/define_expectations/	Data quality expectations.
Great Expectations suites - https://docs.greatexpectations.io/docs/core/define_expectations/organize_expectation_suites/	Organising validation suites.
MLflow tracking - https://mlflow.org/docs/latest/ml/tracking/	Experiment lineage and artefact tracking.
MLflow model registry - https://mlflow.org/docs/latest/ml/model-registry	Model artefact management and lifecycle.
MLflow agent evaluation - https://mlflow.org/docs/latest/genai/eval-monitor/running-evaluation/agents/	Trace scoring and agent-evaluation workflows.
Hugging Face model cards - https://huggingface.co/docs/hub/model-cards	Metadata-rich model documentation.
Hugging Face dataset cards - https://huggingface.co/docs/hub/en/datasets-cards	Dataset cards and YAML metadata.
Google Research model cards - https://research.google/pubs/model-cards-for-model-reporting/	Structured model reporting.
Data Cards paper - https://arxiv.org/abs/2204.01075	Structured dataset documentation.
NIST AI RMF - https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10	Governance framing for trustworthy AI.

Source	Why it mattered
NIST AI RMF Playbook - https://www.nist.gov/itl/ai-risk-management-framework/nist-ai-rmf-playbook	Practical governance playbook.
Microsoft GraphRAG - https://www.microsoft.com/en-us/research/project/graphrag/	Corpus-level graph retrieval.
Microsoft GraphRAG GitHub - https://github.com/microsoft/graphrag	Implementation and cost caveats.
GraphRAG paper - https://arxiv.org/abs/2404.16130	Graph-based RAG for global corpus questions.
RAGAS - https://aclanthology.org/2024.eacl-demo.16/	Retrieval-augmented generation evaluation.
BEIR - https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/65b9eea6e1cc6bb9f0cd2a47751a186f-Abstract-round2.html	Retrieval benchmark thinking.
CRAG - https://arxiv.org/abs/2406.04744	Freshness and dynamic retrieval benchmark ideas.
RepoBench - https://proceedings.iclr.cc/paper_files/paper/2024/hash/d191ba4c8923ed8fd8935b7c98658b5f-Abstract-Conference.html	Repository-level evaluation.
SWE-bench - https://github.com/swe-bench/SWE-bench	Software-engineering task evaluation.
VS Code context engineering guide - https://code.visualstudio.com/docs/copilot/guides/context-engineering-guide	Curated project context for better AI outputs.
GitHub context engineering blog - https://github.blog/ai-and-ml/generative-ai/want-better-ai-outputs-try-context-engineering/	Practical context-engineering framing.
LangSmith evaluation concepts - https://docs.langchain.com/langsmith/evaluation-concepts	Component-aware evaluation and curated examples.
LangChain long-term memory - https://docs.langchain.com/oss/python/langchain/long-term-memory	Namespaced memory patterns.
OpenAI memory compaction cookbook - https://developers.openai.com/cookbook/examples/agents_sdk/building_reliable_agents_memory_compaction	Distinguishing compaction from persistent memory.